

Driver Monitoring System (DMS): מדריך העברת פרויקט למידה עמוקה

מסמך זה מרכז את כל המידע, הארכיטקטורה והקוד שנבנו במהלך השיחה עבור פרויקט הגמר בקורס למידה עמוקה (SCE 26a). המסמך מיועד להזנה ישירה לתוך Claude (בין אם באמצעות Claude Code או כקובץ קונטקסט בתוך ה-Extension של VS Code) כדי להמשיך את הפיתוח בצורה חלקה.

1. תעודת זהות של הפרויקט (Project Overview)

שדה פרויקט	תיאור והגדרה לפרויקט
שם הפרויקט	Deep-DMS: Real-Time Driver Drowsiness Detection using Spatial-Temporal Networks
תחום (Domain)	(Computer Vision / Multimodal (Spatial + Time-Series
סוג המשימה	Sequential Video Classification (סיווג רב-מחלקתי של סדרות עיתיות)
מחלקות לסיווג (Classes)	עירני (Alert), עייף/מנוקד (Drowsy), נרדם (Sleeping), שר (Singing), מוסח (Distracted), מפהק (Yawning)
סביבת עבודה	סביבה מקומית (Local venv) בתוך VS Code, שימוש ב-PyTorch, OpenCV, MediaPipe, Matplotlib

2. המצב הנוכחי של צינור הנתונים (Pipeline)

הצלחנו להקים בהצלחה סביבה מקומית נקייה ב-VS Code המריצה את קובץ ה-Preprocessing הראשון. הקוד קורא בהצלחה סרטון מקומי (במקרה שלנו vid1.mp4 בעל 457 פריימים), מחלץ נקודות ציון באמצעות MediaPipe ומחשב שני מדדים מרכזיים בזמן אמת: EAR (עבור העיניים) ו-MAR (עבור הפה).

הקוד הנוכחי העובד במחשב (main.py):

```
import cv2
import numpy as np
```

```

import matplotlib.pyplot as plt
import mediapipe as mp

mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
)

RIGHT_EYE_INDICES = [33, 160, 158, 133, 153, 144]
LEFT_EYE_INDICES = [362, 385, 387, 263, 373, 380]
MOUTH_INDICES = [78, 82, 312, 308, 317, 87]

def get_coords(landmarks, indices, img_w, img_h):
    return np.array([[landmarks.landmark[i].x * img_w,
landmarks.landmark[i].y * img_h] for i in indices])

ear_values = []
mar_values = []

video_name = 'vid1.mp4'
cap = cv2.VideoCapture(video_name)

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break
    img_h, img_w, _ = frame.shape
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = face_mesh.process(rgb_frame)

    if results.multi_face_landmarks:
        for face_landmarks in results.multi_face_landmarks:
            right_eye = get_coords(face_landmarks,
RIGHT_EYE_INDICES, img_w, img_h)
            left_eye = get_coords(face_landmarks, LEFT_EYE_INDICES,
img_w, img_h)

```

```

        mouth = get_coords(face_landmarks, MOUTH_INDICES,
img_w, img_h)

    def calc_ear(eye):
        v1 = np.linalg.norm(eye[1] - eye[5])
        v2 = np.linalg.norm(eye[2] - eye[4])
        h = np.linalg.norm(eye[0] - eye[3])
        return (v1 + v2) / (2.0 * h)

    def calc_mar(m):
        v1 = np.linalg.norm(m[1] - m[5])
        v2 = np.linalg.norm(m[2] - m[4])
        h = np.linalg.norm(m[0] - m[3])
        return (v1 + v2) / (2.0 * h)

    ear = (calc_ear(right_eye) + calc_ear(left_eye)) / 2.0
    mar = calc_mar(mouth)
    ear_values.append(ear)
    mar_values.append(mar)
else:
    ear_values.append(0)
    mar_values.append(0)

cap.release()
print("Processing complete!")

```

3. כיצד להמשיך את הפרויקט מול Claude ב-VS Code

כדי להתקדם לשלב של בניית המודל בלמידה עמוקה, מומלץ להזין ל-Claude (בצ'אט שלו בתוך ה-IDE) את ההנחיות והפרומפטים הבאים שלב אחר שלב:

שלב א': יצירת קבצי NumPy מסודרים כדאטאסט (Data Engineering)

כדי לאמן מודל PyTorch, אנחנו לא רוצים לקרוא את הוידאו הגולמי כל פעם מחדש. אנחנו צריכים לשמור את רשימות ה-EAR וה-MAR כקבצי מערכים מובנים יחד עם הלייבלים שלהם.
פרומפט מומלץ להעתקה עבור קלוד:

```
"I am building a Driver Monitoring System in PyTorch. I have a
```

script that extracts EAR and MAR values per frame from videos. I want to refactor my preprocessing pipeline so that for each video, it saves a NumPy array (.npy) containing the features [EAR, MAR] of shape (num_frames, 2), and generates a corresponding label file or structured dictionary mapping the video to its multi-class state (Alert, Drowsy, Yawning, Distracted, etc.). Please help me build a clean script to save processed features into a 'data/processed' directory."

שלב ב': בניית מחלקת Dataset ו-DataLoader ב-PyTorch

רשת ה-LSTM צריכה לקבל חלונות זמן קבועים (למשל סדרה של 30 פריימים ברצף). לשם כך נדרשת מחלקה מותאמת אישית של PyTorch שתדע לחתוך את מערכי ה-NumPy באמצעות Sliding Window. פרומפט מומלץ להעתקה עבור קלוד:

"Now that my features are saved as NumPy arrays of shape (num_frames, 2), please write a PyTorch custom Dataset class. The dataset needs to implement a sliding window approach with a sequence_length of 30 frames (representing 1 second of video) and return a tensor of shape (30, 2) and its corresponding multi-class label. Also, include the DataLoader implementation with batching and shuffling for training."

שלב ג': כתיבת רשת ה-LSTM ולולאת האימון (Model & Training)

כתיבת ארכיטקטורת רשת הניירונים הרקורסיבית, הגדרת פונקציית הפסד מסוג CrossEntropyLoss (המתאימה לסיווג רב-מחלקתי) ולולאת אימון מלאה שכוללת מעקב אחרי ה-Loss וה-Accuracy. פרומפט מומלץ להעתקה עבור קלוד:

"Please create a PyTorch LSTM model called `DrowsinessLSTM` that takes an input size of 2 (EAR and MAR) over a sequence length of 30, passes it through a hidden layer (e.g., 64 units), and outputs a multi-class classification layer (e.g., 5 or 6 states). Then, write a standard PyTorch training loop using CrossEntropyLoss and Adam optimizer, including validation tracking and saving the best model weights (.pth)."

4. ארכיטקטורת היעד של תיקיית הפרויקט

כדי לשמור על סדר הנדסי לאורך פיתוח הפרויקט, מומלץ לארגן את הקבצים במבנה הבא אצלך במחשב המקומי:

- **/data/raw_videos** - תיקייה בה תשמרו את סרטוני הוידאו הגולמיים שאתם מקליטים או מורידים (כמו vid1.mp4).
- **/data/processed** - תיקייה בה יישמרו קבצי ה-.npz הקטנים לאחר הרצת שלב ה-Preprocessing.
- **src/dataset.py** - קובץ שיכיל את מחלקת ה-Dataset וה-DataLoader של PyTorch.
- **src/model.py** - קובץ שיכיל את ארכיטקטורת רשת ה-LSTM שלכם.
- **train.py** - הסקריפט הראשי שמריץ את לולאת האימון, מציג גרפים של גרף הלימוד (Loss/Accuracy) ושומר את המודל המאומן.
- **inference.py** - קובץ שיטען את המודל המאומן (.pth) ויפעיל אותו בזמן אמת על מצלמת הרשת (Webcam) של המחשב כדי להציג את התראות העירנות בשידור חי!